

Rapport de veille technologique backend

Baptiste Appriou

01 NestJS

NestJS est un framework backend qui fonctionne avec Node.js et s'appuie sur TypeScript. Il propose une organisation en modules, contrôleurs et services. L'injection de dépendances permet de fournir les services aux classes qui en ont besoin, plutôt que de les créer partout. Cette structure donne des repères communs dans une équipe.

Pour une API Kanban, cette approche aide à séparer les utilisateurs, les listes et les cartes. Les guards peuvent protéger les routes, tandis que les DTO et leur validation encadrent les données reçues. Le module officiel Swagger facilite la documentation. L'accès aux données reste un choix complémentaire : dans ce projet, il passe par Prisma.

En contrepartie, il faut comprendre plusieurs notions avant d'être vraiment à l'aise. Pour une API minuscule, les modules et les décorateurs peuvent sembler encombrants. Le typage TypeScript ne remplace pas non plus la validation des requêtes envoyées par un utilisateur.

Node.js convient aux traitements avec beaucoup d'attentes réseau, mais un calcul JavaScript lourd peut bloquer la boucle événementielle. Il faut alors isoler ce travail. NestJS me semble donc pertinent pour une API structurée, surtout dans une équipe JavaScript, sans être automatiquement le meilleur choix pour tous les projets.

02 Symfony

Symfony est un framework PHP organisé autour de composants et d'un conteneur de services. Il encourage la séparation entre les contrôleurs et la logique métier. Son autowiring utilise les types déclarés pour injecter les services. Les attributs PHP permettent notamment de définir les routes et certaines règles de sécurité près du code.

Pour le Kanban, il offre des outils de validation et de sécurité adaptés à la gestion des utilisateurs. Doctrine peut prendre en charge les relations entre utilisateurs, listes et cartes. Il s'agit d'un outil associé, pas d'une obligation du framework. La documentation Swagger peut être ajoutée avec NelmioApiDocBundle.

Son calendrier de maintenance est un vrai avantage pour un produit suivi plusieurs années. La branche 7.4 LTS prévoit des corrections de bugs jusqu'en novembre 2028 et de sécurité jusqu'en novembre 2029. La branche stable 8.1 propose un autre compromis, davantage orienté vers les nouveautés.

La difficulté vient surtout de l'apprentissage de PHP, de la configuration et des outils associés lorsque l'équipe ne les connaît pas. Les performances demandent aussi une configuration de production correcte, notamment avec OPcache. Je retiendrais facilement Symfony pour une équipe PHP ; dans une équipe TypeScript, le changement d'environnement doit apporter un bénéfice suffisamment clair.

03 Spring Boot

Spring Boot facilite la création d'applications basées sur Spring, généralement en Java ou Kotlin. Ses starters regroupent les dépendances utiles et son auto-configuration adapte le démarrage aux bibliothèques présentes. L'application peut embarquer son serveur HTTP. Cela évite une partie de la configuration initiale, sans supprimer le besoin de comprendre Spring.

Pour le Kanban, on peut organiser le code en contrôleurs, services et accès aux données. L'écosystème comprend Spring Security pour la sécurité et Spring Data pour la persistance. Actuator apporte des fonctions de suivi en production. Swagger peut être intégré avec springdoc-openapi, un projet complémentaire dont il faut vérifier la compatibilité.

Cette richesse est intéressante pour une équipe Java ou un produit avec plusieurs intégrations métier. En revanche, apprendre simultanément Java, Spring et Maven ou Gradle augmente le travail initial. L'auto-configuration peut aussi rendre certaines erreurs moins évidentes à comprendre au début.

Spring MVC et WebFlux proposent des modèles d'exécution différents. Le second est non bloquant, mais il ne rend pas automatiquement une application plus rapide. La mémoire et le démarrage doivent être mesurés avec une configuration précise. Pour ce petit projet, je ne choisirais donc pas Spring Boot uniquement pour sa réputation de framework d'entreprise : les compétences disponibles comptent aussi.

04 Pourquoi je choisis NestJS

J'ai choisi NestJS pour construire une API Kanban structurée tout en restant dans l'écosystème JavaScript et TypeScript. Mon objectif n'est pas de montrer qu'il surpasse les deux autres frameworks. Je cherche surtout un compromis entre la prise en main, l'organisation du code et les besoins du projet.

Le dépôt illustre ce choix : les modules séparent déjà l'authentification, les utilisateurs et les listes. Les contrôleurs gèrent les routes et délèguent les traitements aux services. La validation globale contrôle les entrées, et Swagger expose la documentation sur /api. Ce découpage me paraît plus facile à expliquer et à faire reprendre par un autre développeur.

Je garde toutefois plusieurs limites en tête. Les services utilisent directement Prisma, ce qui crée un couplage avec cet outil. Il faut aussi suivre les versions de Node.js et des dépendances. Enfin, NestJS ne garantit ni la sécurité ni les performances : les contrôles de propriété et les requêtes doivent être testés.

Enfin dans un cadre plus personnel, j'ai eu envie de choisir NestJs car c'est le framework qui se rapprochait le plus de ce que je fais en entreprise. De plus, la découverte de Prisma m'intéresse afin de pouvoir apporter ce plus à mon entreprise. La courbe d'apprentissage me semble rude mais je pense que ça ira sur la durée.